

PORTFOLIO PLATFORM

Product Design & Technical Documentation

Full-Stack Developer · IT Specialist · Technical Writer

Next.js 16 · Prisma ORM · PostgreSQL · Shadcn/ui · JWT Auth

Version 1.0 · 16 March 2026

Table of Contents

Table of Contents.....	2
1. Platform Overview.....	4
1.1 Platform Identity.....	4
1.2 Joint Name — Projects & Blog.....	4
1.3 Four Public Pages at a Glance	4
2. Home Page (/).....	6
2.1 Sections & Content Spec.....	6
2.1.1 Hero	6
2.1.2 About Me.....	6
2.1.3 Featured Work (4-card grid)	6
2.1.4 Skills & Stack	7
2.1.5 Contact Section.....	7
3. Lab Page (/lab)	8
3.1 Index Page (/lab)	8
3.2 Project Content Spec.....	8
3.3 Article / Blog Post Content Spec.....	9
3.3.1 Suggested Seed Articles	9
3.4 Detail Page (/lab/[slug])	10
4. Services Page (/services).....	11
4.1 Service Offerings (4 cards)	11
Service Model (Prisma).....	11
4.2 Client Workflow (5-stage process)	11
4.3 Testimonials	12
4.4 Case Studies Section (/services#case-studies)	12
4.5 Contact / Booking CTA	13
5. Dashboard (/dashboard)	14
5.1 Authentication.....	14
5.2 Dashboard Routes & CRUD Scope	14
5.3 SiteSettings Model (Single Row)	15
6. Full Prisma Schema	17
6.1 All Models.....	17
User	17
Project	17
BlogPost	17
Service, Testimonial, WorkflowStep, Skill, Certification, Experience, Education.....	17

ContactMessage, NewsletterSubscriber, SiteSettings	18
Enums	18
7. Project File Structure.....	19
8. Brand & Design System	20
8.1 Color Palette.....	20
8.2 Typography	20
8.3 Animation Strategy	20
8.4 Dark / Light Mode	21
9. SEO & Additional Features	22
9.1 SEO Implementation	22
9.2 Analytics	22
9.3 Newsletter	22
9.4 Downloadable CV	23
9.5 Search	23
10. Future Roadmap — Client Portal (v2.0)	24
10.1 What the Portal Will Do.....	24
10.2 New Prisma Models for v2.0	24
10.3 New Routes for v2.0	25
11. Setup & Quick Start.....	26
11.1 Install Commands.....	26
11.2 Environment Variables (.env.local)	26
11.3 First Run Checklist	26

1. Platform Overview

This document is the single source of truth for designing, building, and maintaining the personal portfolio and freelance services platform. It covers content strategy, page specifications, CRUD mappings, dashboard architecture, Prisma schema, project file structure, and the future client portal roadmap.

1.1 Platform Identity

Platform name	Portfolio + Freelance Platform (public domain TBD)
Owner	CS Bachelor graduate — Full-Stack Developer & IT Specialist
Tech stack	Next.js 16 (App Router), Prisma ORM, PostgreSQL, Shadcn/ui, Tailwind CSS, JWT + bcryptjs auth, Framer Motion
Hosting target	Vercel (frontend + serverless API routes) + Neon or Supabase (PostgreSQL)
Primary audience	Recruiters, potential clients, tech community, employers
Version	1.0 — MVP (portfolio + services). v2.0 = Client Portal.

1.2 Joint Name — Projects & Blog

Personal code projects and written articles/studies are unified under a single section. This replaces the generic "Work" label and conveys the research-lab, academic-publication aesthetic that was requested.

Chosen name: "Lab" — route: /lab

Rationale: short, tech-native, implies experimentation, research, and craft — equally appropriate for a code project and a written study.

Tagline suggestion: "Experiments, studies & builds"

URL pattern: /lab (index) · /lab/[slug] (individual item)

Alt names considered: /works, /research, /index, /notebook — "Lab" was selected for its versatility and brand fit.

Content in /lab is split by type tag at the UI level (filter: All | Projects | Articles | Case Studies), but lives in a single shared route. Client projects are NOT in /lab — they belong to the Services page.

1.3 Four Public Pages at a Glance

Page	Route	Analogy / Feel	Primary CTA
Home	/	LinkedIn profile + résumé — scannable overview	Hire me / View Lab
Lab	/lab	Research lab publications — detailed, citable work	Read / View repo
Services	/services	Freelance profile (Upwork/Fiverr style) — without filler	Book a call / Contact
Dashboard	/dashboard	Internal CMS — every byte of content is editable	CRUD all content

2. Home Page (/)

The home page functions as a living résumé and LinkedIn profile. It is a single vertical scroll with anchored sections. Every piece of text, every link, every image, and even the owner's profile details are editable from the dashboard. Nothing is hardcoded in the source.

2.1 Sections & Content Spec

2.1.1 Hero

Layout	Two-column: photo left, text right (stacks on mobile)
Photo	Professional headshot — stored via UploadThing, URL in SiteSettings.heroImageUrl
Headline	First name + surname from SiteSettings.ownerName
Role tagline	e.g. "Full-Stack Developer · IT Specialist · Based in Yaoundé" — SiteSettings.tagline
Sub-tagline	1–2 sentence value proposition — SiteSettings.heroSubtext
Availability badge	"Open to work" green dot — toggled via SiteSettings.availableForWork (Boolean)
CTA buttons	"View Lab" → /lab "Hire Me" → /services "Download CV" → /api/cv
Animation	Framer Motion: text fades up on mount, photo scales in from 0.95

2.1.2 About Me

Biography	Long-form HTML stored in SiteSettings.aboutBio — Tiptap editor in dashboard
Education	Academic degrees, certifications — each as an Education record (CRUD from dashboard)
Experience	Work history — each as an Experience record (CRUD from dashboard)
Interests	Tags list — SiteSettings.interests (comma-separated string)
CV download	PDF link — SiteSettings.cvUrl (file uploaded via dashboard)

2.1.3 Featured Work (4-card grid)

This section is the only place the home page shows content from the Lab and Services data. It does not duplicate full content — it shows four curated preview cards.

Slot	Source model	How selected	Card shows
Top personal project 1	Project (type: PERSONAL)	featuredRank = 1 in dashboard	Title, stack tags, 1-line summary, GitHub + demo links
Top personal project 2	Project (type: PERSONAL)	featuredRank = 2 in dashboard	Same as above
Top article	BlogPost	featuredOnHome = true in dashboard	Title, category, read-time, excerpt
Top client project	Project (type: CLIENT)	featuredRank = 1 (client) in dashboard	Title, industry tag, outcome metric, case study link

i The "top client project" card on the home page links to /services#case-studies — it is a teaser, not a full case study hosted in /lab.

2.1.4 Skills & Stack

Categories	Frontend, Backend, DevOps, IT Support, Tools, Languages
Each skill record	name, category, proficiency (1–5), iconSlug (for devicons), order
Display	Category tabs — each showing icon grid with proficiency bar or dots
Certifications	Separate Certification model: name, issuer, date, credentialUrl, logoUrl
CRUD	Full CRUD from /dashboard/skills and /dashboard/certifications

2.1.5 Contact Section

Form fields	Name, Email, Subject (dropdown), Message — validated with Zod
Submission	Server Action → creates ContactMessage record in DB → Resend email to owner
Social links	GitHub, LinkedIn, Twitter/X, email — stored in SiteSettings.socials (JSON)
Phone	SiteSettings.phone — optional, shown/hidden via SiteSettings.showPhone (Boolean)
Location	SiteSettings.location — "Yaoundé, Cameroon"
Response time	SiteSettings.responseTime — e.g. "Usually within 24 hours"

3. Lab Page (/lab)

The Lab is the intellectual showcase of the platform. It presents personal code projects and written articles/studies side by side, styled like publications from a research lab — each item citable, detailed, and evidence-based. There are no client projects here; those live in /services.

3.1 Index Page (/lab)

Filter tabs	All Projects Articles Case Studies
Sort	Recent (default) · Most viewed · Alphabetical — client-side via URL param
Search	Fuse.js fuzzy search across title + tags + excerpt (client-side)
Card grid	2-column on desktop, 1-column on mobile — masonry optional
Pagination	Cursor-based (Prisma cursor pagination) — 9 items per page
Newsletter CTA	Footer of index: email input → NewsletterSubscriber record + confirmation email

3.2 Project Content Spec

Each project is a richly documented build. The goal is to read like a technical case study, not just a README card.

Field	Type	Description
title	String	Project name — displayed as H1 on detail page
slug	String @unique	Auto-generated from title — used in /lab/[slug]
type	Enum: PERSONAL CLIENT	CLIENT items only shown in /services, not /lab
summary	String (160 chars max)	One-liner for cards and SEO meta description
content	String @db.Text	Full MDX/HTML body — Tiptap editor — problem, solution, architecture, lessons
stack	String[]	Tech tags: ["Next.js", "PostgreSQL", "Prisma"] — displayed as badges
coverImageUrl	String?	Hero image — UploadThing URL
screenshotUrls	String[]	Gallery images (up to 6)
demoUrl	String?	Live demo link
repoUrl	String?	GitHub repository link

Field	Type	Description
status	Enum: LIVE WIP ARCHIVED	Shown as a status badge on the card
featuredRank	Int?	1 or 2 for personal, 1 for client = shown on home page
featured	Boolean	Pinned to top of /lab index
views	Int @default(0)	Incremented on page load via Server Action
publishedAt	DateTime?	Null = draft; set to publish
createdAt / updatedAt	DateTime	Auto-managed by Prisma

3.3 Article / Blog Post Content Spec

Field	Type	Description
title	String	Article title — H1 on detail page
slug	String @unique	URL-safe, auto-generated
category	Enum: TUTORIAL RESEARCH REFLECTION ROADMAP TRENDS CHEATSHEET	Shown as colored badge
excerpt	String (280 chars)	Card preview and meta description
content	String @db.Text	Full article body — Tiptap with code block support (Shiki highlighting)
coverImageUrl	String?	Article hero image
tags	String[]	Topic tags: ["AI", "React", "Cloud"]
readTime	Int	Minutes — auto-calculated from content length on save
featuredOnHome	Boolean @default(false)	Shows in the "top article" slot on home page
featured	Boolean	Pinned at top of /lab index
views	Int @default(0)	View counter
publishedAt	DateTime?	Null = draft

3.3.1 Suggested Seed Articles

- "How to Build a Full-Stack App with Next.js 16 and Prisma" — Tutorial

- "The Future of AI: Trends for 2025–2026" — Research / Trends
- "A Beginner's Guide to Cloud Computing" — Tutorial
- "My Journey from CS Student to Full-Stack Developer" — Reflection
- "Setting Up a Home Lab for IT Practice" — Tutorial / IT Support
- "Web Performance Optimization Cheatsheet" — Cheatsheet

3.4 Detail Page (/lab/[slug])

Header	Cover image, title, category badge, read-time (articles) or status badge (projects), publish date
Stack badges	Colored pills for each tech tag — links to /lab?stack=tag filter
Body	Full rendered MDX/HTML with syntax-highlighted code blocks (Shiki), images, and embeds
Links	Demo + GitHub buttons for projects; external reference links for articles
Share	Copy link + Twitter/X share button
Related	3 items with same tags (Prisma query: same stack[] overlap)
SEO	generateMetadata() with full OG tags; /api/og dynamic image
Analytics	View count incremented via Server Action (debounced, once per session)

4. Services Page (/services)

The services page is modelled after a premium freelance profile — the credibility of Upwork or Toptal, without the platform boilerplate. Visitors are potential clients. The goal is to answer three questions in under 30 seconds: What can you do? How do you work? Have others trusted you?

4.1 Service Offerings (4 cards)

Each service is a record in the database, fully editable from the dashboard. The four default services:

Service	Sub-items (bullets on card)	Icon slug
Web Development	Business websites · Web applications · CMS solutions · E-commerce	code-2
IT Support & Consulting	Computer troubleshooting · Network administration · System maintenance · Security audits	server
Digital Transformation	Business automation · Custom software · Database management · Cloud migration	cpu
Technical Writing	Blog posts · Tutorials · API documentation · User manuals	file-text

Service Model (Prisma)

id	cuid — primary key
title	Service name (e.g. "Web Development")
description	Short paragraph — shown above bullets
bullets	String[] — sub-items listed on card
iconSlug	Lucide icon name — rendered client-side
order	Int — controls card display order
published	Boolean — show/hide without deleting

4.2 Client Workflow (5-stage process)

The workflow section shows how you collaborate with clients, removing ambiguity from the hiring decision.

Stage	Label	Detail shown to visitor	Editable field
01	Discovery	Initial call · Scope document · Requirements analysis	WorkflowStep.description

Stage	Label	Detail shown to visitor	Editable field
02	Proposal	Fixed-price or hourly quote · Milestone timeline · Contract	WorkflowStep.description
03	Build	Agile sprints · Weekly updates · Staging previews	WorkflowStep.description
04	Review	Up to 3 feedback rounds · QA testing · Revisions	WorkflowStep.description
05	Deliver	Launch · Handover documentation · 30-day follow-up support	WorkflowStep.description

i WorkflowStep is its own Prisma model so steps can be reordered, renamed, or replaced entirely from the dashboard.

4.3 Testimonials

Testimonials are the most influential trust signal on a freelance page. Every field is stored in the DB and managed from the dashboard. The published flag allows collecting testimonials privately before surfacing them.

Field	Type	Notes
id	String @id @default(cuid())	
clientName	String	Full name of the client
clientRole	String	e.g. "CEO" or "Lead Engineer"
company	String	Company or organisation name
quote	String @db.Text	The testimonial text — no length limit
stars	Int @default(5)	1–5 star rating — displayed as star icons
avatarUrl	String?	Optional headshot — UploadThing URL
projectRef	String?	Free-text project reference shown on card
date	DateTime	Date of testimonial
published	Boolean @default(false)	Must be set true to appear on /services
order	Int @default(0)	Controls display order in grid

4.4 Case Studies Section (/services#case-studies)

Client projects that the owner chooses to feature are displayed here. They are Project records with type: CLIENT and published: true. The card shows a teaser; there is no full detail page on /services — clients click through if they want more.

Card content	Project title, client industry tag, 1-line outcome ("Reduced load time by 60%"), tech stack, link to case study PDF or /lab/[slug]
---------------------	--

Data source	Project model filtered by type = CLIENT AND published = true
Max shown	6 — paginated if more exist
CRUD	Managed via /dashboard/projects — same model as personal projects, differentiated by type enum

4.5 Contact / Booking CTA

At the bottom of /services, a dedicated CTA section replaces the generic contact form with a services-focused one.

Form fields	Name, Email, Service type (dropdown from Service.title), Budget range (dropdown), Project description (textarea), Timeline (dropdown)
Submission	Server Action → ContactMessage record with source = "SERVICES" tag → Resend email to owner
Calendly embed	Optional — SiteSettings.calendlyUrl — shows "Book a discovery call" button if set

5. Dashboard (/dashboard)

The dashboard is a private, JWT-protected CMS. Every single piece of content displayed on the public site — including the owner's own name, photo, bio, and availability status — is editable here. No content is hardcoded anywhere in the Next.js source.

5.1 Authentication

Method	Custom JWT — no NextAuth
Password storage	bcryptjs hash stored in User.passwordHash
Token	jose-signed JWT stored in httpOnly cookie (name: portfolio_session)
Middleware	middleware.ts at root matches /dashboard/:path* — redirects to /login if no valid token
Session duration	7 days — refreshed on each dashboard visit
Login page	/login — email + password form
Logout	Server Action: clears cookie, redirects to /login
Multi-user	Single User record in v1 — admin only. Role field reserved for v2.

5.2 Dashboard Routes & CRUD Scope

Route	Manages	Operations
/dashboard	Overview stats	Read: total projects, posts, messages, views, subscribers
/dashboard/projects	Project index	List, filter by type/status, bulk publish/unpublish
/dashboard/projects/new	New project	Create with Tiptap editor, image upload, tag picker
/dashboard/projects/[id]/edit	Edit project	Full update including featured rank, published status
/dashboard/blog	Blog post index	List, filter by category/status, bulk operations
/dashboard/blog/new	New article	Create with Tiptap, auto read-time, tag picker
/dashboard/blog/[id]/edit	Edit article	Full update including featuredOnHome toggle
/dashboard/services	Services page	CRUD: service cards, workflow steps, testimonials, case study refs

Route	Manages	Operations
/dashboard/skills	Skills & certifications	CRUD: skills with proficiency, certs with dates and URLs
/dashboard/experience	Work history & education	CRUD: Experience and Education records
/dashboard/messages	Contact messages	Read, mark as read, reply via Resend, delete
/dashboard/subscribers	Newsletter list	Read, export CSV, delete/unsubscribe
/dashboard/settings	Site-wide settings	Update SiteSettings: name, tagline, bio, photo, CV, socials, availability

5.3 SiteSettings Model (Single Row)

There is exactly one SiteSettings row in the database (id = "singleton"). The dashboard Settings page reads and updates this row. It stores everything that belongs to "the owner" as a person.

Field	Type	Used on
ownerName	String	Home hero, navbar, footer, meta author
tagline	String	Home hero sub-headline
heroSubtext	String	Home hero paragraph
heroImageUrl	String?	Home hero photo
aboutBio	String @db.Text	Home About section — full HTML
location	String	Home contact section
email	String	Home contact, footer
phone	String?	Home contact (if showPhone = true)
showPhone	Boolean	Toggle phone visibility
cvUrl	String?	Download CV button
availableForWork	Boolean	Availability badge on hero
calendlyUrl	String?	Services page booking CTA
responseTime	String	Contact section label
interests	String	About section — comma-separated
socials	Json	{ github, linkedin, twitter, youtube } — all optional
metaDescription	String	Fallback SEO description for all pages
googleAnalyticsId	String?	GA4 measurement ID — injected into root layout

Field	Type	Used on
newsletterEnabled	Boolean	Show/hide newsletter signup on /lab

6. Full Prisma Schema

Schema file: prisma/schema.prisma

Database: PostgreSQL (provider = "postgresql")

All models use cuid() as primary key for URL-safe IDs.

Enums are defined at the bottom of the schema.

Run: `npx prisma migrate dev --name init` after first setup.

6.1 All Models

User

```
model User {
  id          String    @id @default(cuid())
  email       String
  @unique     passwordHash String
  name        String    role      Role
  @default(ADMIN) createdAt    DateTime @default(now())
  updatedAt   DateTime @updatedAt }
```

Project

```
model Project {
  id          String    @id @default(cuid())
  title       String
  slug        String    @unique
  type        ProjectType
  @default(PERSONAL)
  summary     String    @db.VarChar(160)
  content     String
  @db.Text    stack      String[]
  coverImageUrl String?
  screenshotUrls String[]
  demoUrl     String?   repoUrl   String?
  status      ProjectStatus @default(WIP)
  featuredRank Int?     featured   Boolean
  @default(false)
  views       Int       @default(0)
  publishedAt DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt }
```

BlogPost

```
model BlogPost {
  id          String    @id @default(cuid())
  title       String
  slug        String    @unique
  category     PostCategory
  excerpt     String    @db.VarChar(280)
  content     String
  @db.Text    coverImageUrl String?
  tags        String[]
  readTime    Int
  @default(0)
  featuredOnHome Boolean @default(false)
  featured    Boolean
  @default(false)
  views       Int       @default(0)
  publishedAt DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt }
```

Service, Testimonial, WorkflowStep, Skill, Certification, Experience, Education

```
model Service {
  id          String    @id @default(cuid())
  title       String
  description String
  bullets     String[]
  iconSlug    String
  order       Int
  @default(0)
  published   Boolean @default(true) }
model Testimonial {
  id          String    @id @default(cuid())
  clientName  String
  clientRole  String
  company     String
  quote       String    @db.Text
  stars       Int       @default(5)
  avatarUrl   String?
  projectRef  String?
  date        DateTime
  published   Boolean @default(false)
  order       Int       @default(0) }
model WorkflowStep {
  id          String    @id @default(cuid())
  stepNumber  Int
  label       String
  description String
  order       Int       @default(0) }
model Skill {
  id          String    @id @default(cuid())
  name        String
  category    SkillCategory
  proficiency Int
  @default(3)
  iconSlug    String?
  order       Int }
```

```

@default(0) } model Certification { id String @id @default(cuid())
name String issuer String issuedAt DateTime expiresAt
DateTime? credentialUrl String? logoUrl String? } model Experience {
id String @id @default(cuid()) company String role String
description String @db.Text startDate DateTime endDate DateTime?
current Boolean @default(false) order Int @default(0) } model
Education { id String @id @default(cuid()) institution String
degree String field String startYear Int endYear Int?
current Boolean @default(false) order Int @default(0) }

```

ContactMessage, NewsletterSubscriber, SiteSettings

```

model ContactMessage { id String @id @default(cuid()) name
String email String subject String message String @db.Text
source MessageSource @default(HOME) read Boolean @default(false)
createdAt DateTime @default(now()) } model NewsletterSubscriber { id
String @id @default(cuid()) email String @unique confirmed Boolean
@default(false) createdAt DateTime @default(now()) } model SiteSettings { id
String @id @default("singleton") ownerName String @default("Your Name")
tagline String @default("") heroSubtext String @default("")
heroImageUrl String? aboutBio String @db.Text @default("")
location String @default("") email String @default("")
phone String? showPhone Boolean @default(false) cvUrl
String? availableForWork Boolean @default(true) calendlyUrl String?
responseTime String @default("Within 24 hours") interests String
@default("") socials Json @default("{}") metaDescription String
@default("") googleAnalyticsId String? newsletterEnabled Boolean @default(true)
}

```

Enums

```

enum Role { ADMIN CLIENT } enum ProjectType { PERSONAL CLIENT } enum
ProjectStatus { LIVE WIP ARCHIVED } enum PostCategory { TUTORIAL RESEARCH
REFLECTION ROADMAP TRENDS CHEATSHEET } enum SkillCategory { FRONTEND BACKEND DEVOPS
IT_SUPPORT TOOLS LANGUAGES } enum MessageSource { HOME SERVICES }

```

7. Project File Structure

```

portfolio/ | | prisma/ | | schema.prisma | | seed.ts | | migrations/ | |
public/ | | (static assets, OG fallback image) | | src/ | | app/ | |
layout.tsx | | Root layout - fonts, ThemeProvider, Analytics | |
page.tsx | | Home page - all sections as RSCs | | globals.css | |
| | not-found.tsx | | loading.tsx | | lab/ | |
page.tsx | | /lab index - filter, search, pagination | | [slug]/ | |
| | page.tsx | | /lab/[slug] - project or article detail | |
| | services/ | | page.tsx | | /services - service cards,
workflow, testimonials | | login/ | | page.tsx
Login form | | dashboard/ | | layout.tsx | | Auth
guard + sidebar layout | | page.tsx | | Dashboard overview / stats
| | projects/ | | page.tsx | | new/page.tsx | |
| | [id]/edit/page.tsx | | blog/ | | page.tsx | |
| | new/page.tsx | | [id]/edit/page.tsx | |
services/page.tsx | | CRUD: service cards, workflow, testimonials | |
skills/page.tsx | | experience/page.tsx | | messages/page.tsx | |
| | subscribers/page.tsx | | settings/page.tsx | |
api/ | | auth/ | | login/route.ts | |
logout/route.ts | | og/route.tsx | | Dynamic OG image generation | |
| | cv/route.ts | | CV download (redirect or stream) | |
upload/route.ts | | UploadThing handler | | components/ | | ui/
Shadcn components (button, card, dialog, etc.) | | sections/
Home page sections | | Hero.tsx | | About.tsx | |
FeaturedWork.tsx | | Skills.tsx | | Contact.tsx | | lab/
Lab page components | | LabCard.tsx | | LabFilter.tsx | |
| | LabSearch.tsx | | services/ | | Services page components | |
| | ServiceCard.tsx | | Workflow.tsx | | TestimonialCard.tsx
| | dashboard/ | | Dashboard UI components | |
Sidebar.tsx | | StatsCard.tsx | | DataTable.tsx | |
RichEditor.tsx | | Tiptap wrapper | | ImageUpload.tsx | |
shared/ | | Navbar.tsx | | Footer.tsx | |
ThemeToggle.tsx | | AvailabilityBadge.tsx | | lib/ | |
prisma.ts | | Prisma client singleton | | auth.ts
JWT helpers (sign, verify, getSession) | | uploadthing.ts | | email.ts
Resend helpers | | search.ts | | Fuse.js setup | | utils.ts
cn(), slugify(), readTime() | | validators.ts | | Zod schemas for all
forms | | constants.ts | | Nav links, category labels, color maps | |
| | actions/ | | Next.js Server Actions | | projects.ts | |
| | blog.ts | | services.ts | | settings.ts | |
| | contact.ts | | auth.ts | | types/ | | index.ts
Re-exports | | db.ts | | Prisma type aliases | | api.ts
API response types | | middleware.ts | | Auth guard for /dashboard/*
| | next.config.ts | | tailwind.config.ts | | .env.local | | package.json

```

8. Brand & Design System

8.1 Color Palette

Token	Hex	Tailwind alias	Usage
Midnight	#0A0F1E	brand-midnight	Page background (dark mode)
Deep Slate	#111827	brand-surface	Card and section surfaces
Indigo	#4F46E5	brand-indigo	Primary brand color, active nav, section H2 borders
Cyan	#06B6D4	brand-cyan	All CTA buttons (Hire me, Send, Download)
Violet	#8B5CF6	brand-violet	Tech stack badges and category tags
Emerald	#10B981	brand-emerald	Availability status, success states
Snow	#F1F5F9	brand-snow	Primary body text on dark backgrounds
Slate 400	#94A3B8	brand-muted	Secondary / muted text, captions

8.2 Typography

Primary font	Inter or Geist Sans — loaded via next/font/google
Mono font	Geist Mono or JetBrains Mono — code blocks only
H1	56px / 700 — hero only
H2	32px / 600 — section headings
H3	22px / 600 — sub-sections
Body	16px / 400 — line-height 1.7
Small / caption	13px / 400 — dates, tags, meta

8.3 Animation Strategy

- Framer Motion: `whileInView + initial={{ opacity:0, y:24 }}` for section entrances
- `viewport={{ once: true }}` — no replay on scroll-back
- Service cards: `whileHover={{ y: -4 }}` — 200ms ease
- Hero tagline: `staggerChildren` on word spans — 40ms stagger
- Navbar: `backdrop-filter blur` toggled by scroll listener (CSS transition 200ms)
- Buttons: 150ms color transition on hover, `scale(0.97)` on active
- CSS only for micro-interactions — Framer Motion only for layout animations

8.4 Dark / Light Mode

Library	next-themes — wrap root layout in ThemeProvider
Strategy	darkMode: "class" in tailwind.config — dark: prefix everywhere
Storage	localStorage via next-themes (automatic)
Toggle	Sun / moon icon in navbar — useTheme() hook
Default	Dark mode as default (system preference check first)

9. SEO & Additional Features

9.1 SEO Implementation

Feature	Implementation
Page metadata	generateMetadata() in every page.tsx — reads from DB or SiteSettings
Dynamic OG images	/api/og route using next/og — params: title, type, coverImage
Sitemap	src/app/sitemap.ts — MetadataRoute.Sitemap — fetches all slugs from Prisma
robots.txt	src/app/robots.ts — blocks /dashboard/* and /login
Canonical URLs	Set in generateMetadata() alternates.canonical
JSON-LD	Person schema on home page, Article schema on /lab/[slug], Service schema on /services
Image optimization	All images via next/image with explicit width/height — no layout shift
Performance	RSC streaming, Suspense boundaries, next/font (no FOUT), lazy Framer Motion import

9.2 Analytics

Primary	@vercel/analytics — add <Analytics /> to root layout — no cookie banner needed
Secondary (optional)	Google Analytics 4 — GA ID stored in SiteSettings.googleAnalyticsId — injected via next/script strategy="afterInteractive"
View counting	Project and BlogPost views field incremented by Server Action on page load (once per session via sessionStorage flag)

9.3 Newsletter

Trigger	Email input in /lab index footer and optionally /services
Flow	Submit → NewsletterSubscriber record (confirmed=false) → Resend confirmation email → Click link → confirmed=true
Dashboard	/dashboard/subscribers — table with email, date, confirmed status — CSV export button
Unsubscribe	/unsubscribe?token=[jwt] — Server Action sets confirmed=false or deletes record

9.4 Downloadable CV

Option A (static)	Upload PDF to UploadThing → store URL in SiteSettings.cvUrl → button links directly
Option B (dynamic)	@react-pdf/renderer Server Action reads Prisma data → generates PDF on demand → always up-to-date
Recommendation	Start with Option A (faster). Migrate to B when experience section is fully populated.

9.5 Search

Scope	/lab index — searches title, excerpt/summary, tags
Library	Fuse.js v7 — client-side fuzzy search
Data source	Published items fetched at build time into a static JSON file via generateStaticParams — revalidated every 60s
UX	Search input in /lab header — results update on type — no page reload

10. Future Roadmap — Client Portal (v2.0)

The current architecture (v1.0) is intentionally designed to support a client portal without major refactoring. The User model already has a Role enum (ADMIN | CLIENT). The Project model is already shared between personal and client types. The following additions constitute v2.0.

10.1 What the Portal Will Do

Clients will log in at /client and see a private dashboard showing only their project(s). The experience mirrors what freelance platforms like Upwork or Contra provide — but hosted on your own domain, with your own branding.

Feature	Description
Client login	/client/login — separate from owner /login — same JWT mechanism, Role = CLIENT
Project overview	Client sees their project's current status, phase, and percentage completion
Milestone tracker	Ordered list of milestones — each with status (Pending / In Progress / Done) and due date
File delivery	Owner uploads deliverables — client can download from their portal
Messages	Per-project threaded messaging between owner and client — stored in DB
Invoice view	Read-only view of invoice records (amount, status, due date)
Approval requests	Owner marks a milestone as "needs approval" — client can approve or request revision
Activity feed	Timeline of project events: "Design delivered", "Revision requested", "Invoice sent"

10.2 New Prisma Models for v2.0

```
model ClientProject {
  id          String      @id @default(cuid())
  project     Project      @relation(fields: [projectId], references: [id])
  clientId    String       @relation(fields: [clientId], references: [id])
  client      User
  phase       ProjectPhase @default(DISCOVERY)
  progress    Int          @default(0) // 0-100
  milestones  Milestone[]
  messages    Message[]
  files       DeliveryFile[]
  invoices    Invoice[]
}
model Milestone {
  id          String      @id @default(cuid())
  clientProjectId String
  clientProject ClientProject @relation(...)
  title       String
  description String?
  dueDate     DateTime?
  status      MilestoneStatus @default(PENDING)
  order       Int
  approvedAt  DateTime?
}
model Message {
  id          String      @id @default(cuid())
  clientProjectId String
  senderId    String
  body        String
  read        Boolean @default(false)
  createdAt   DateTime
}
model Invoice {
  id          String      @id @default(cuid())
  clientProjectId String
  amount      Decimal
  currency    String      @default("USD")
  status      InvoiceStatus @default(PENDING)
  dueDate     DateTime
  paidAt      DateTime?
}
enum ProjectPhase { DISCOVERY PROPOSAL BUILD REVIEW DELIVERED }
enum MilestoneStatus { PENDING IN_PROGRESS DONE NEEDS_APPROVAL }
enum InvoiceStatus { PENDING PAID OVERDUE }
```


10.3 New Routes for v2.0

Route	Access	Purpose
/client	CLIENT role	Client portal home — project list
/client/[projectId]	CLIENT role	Project overview, milestones, activity feed
/client/[projectId]/files	CLIENT role	Download deliverables
/client/[projectId]/messages	CLIENT role	Threaded messages with owner
/client/[projectId]/invoices	CLIENT role	Invoice view + payment links
/dashboard/clients	ADMIN role	Client list — create client account, assign to project
/dashboard/clients/[id]	ADMIN role	Manage client's project, milestones, files, invoices

Migration note — v1.0 → v2.0

No existing tables are dropped. The User model gains a clientProjects relation.

The Project model gains a ClientProject relation (optional — not all projects have a client).

Run: `npx prisma migrate dev --name add_client_portal`

Middleware update: add `/client/:path*` guard checking `Role = CLIENT` (separate from ADMIN check).

Create first client account via `/dashboard/clients` → "Add client" → system sends invitation email.

11. Setup & Quick Start

11.1 Install Commands

```
# Bootstrap npx create-next-app@latest portfolio --typescript --tailwind --app --
src-dir # Core dependencies npm install prisma @prisma/client bcryptjs jose npm
install @tiptap/react @tiptap/starter-kit @tiptap/extension-code-block-lowlight npm
install framer-motion npm install uploadthing @uploadthing/react npm install resend
zod react-hook-form @hookform/resolvers npm install fuse.js next-themes npm install
lucide-react clsx tailwind-merge npm install shiki # syntax highlighting # Shadcn
UI init npx shadcn@latest init npx shadcn@latest add button card badge dialog table
form input textarea npx shadcn@latest add sheet dropdown-menu avatar separator
switch toast tabs # Prisma npx prisma init # Set DATABASE_URL in .env.local, then:
npx prisma migrate dev --name init npx prisma db seed
```

11.2 Environment Variables (.env.local)

```
DATABASE_URL="postgresql://..." JWT_SECRET="your-256-bit-secret-here"
UPLOADTHING_SECRET="..." UPLOADTHING_APP_ID="..." RESEND_API_KEY="..."
EMAIL_FROM="you@yourdomain.com" NEXT_PUBLIC_APP_URL="https://yourdomain.com"
NEXT_PUBLIC_VERCEL_ANALYTICS_ID="" # auto-set on Vercel
```

11.3 First Run Checklist

1. Run migrations: `npx prisma migrate dev --name init`
2. Seed the DB: `npx prisma db seed` — creates admin user + SiteSettings singleton row
3. Log into /login with seed credentials → change password immediately in /dashboard/settings
4. Update SiteSettings: name, tagline, photo, bio, CV, social links
5. Create at least 2 projects and 1 blog post — set published = true
6. Set featuredRank on home-page items
7. Configure Resend domain for outbound email
8. Add Google Analytics ID to SiteSettings if desired
9. Deploy to Vercel — set all env vars in Vercel dashboard
10. Submit sitemap to Google Search Console: `yourdomain.com/sitemap.xml`

— End of Document —